# RutaSmart — Manual Técnico

## Estructura, componentes y mantenimiento

Sarah Isabella Peña Cortes

Docente: Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

# Tabla de contenido

Información del documento ..... 3

1. Propósito ..... 4

2. Estructura del proyecto ..... 5

3. Componentes principales ..... 6

4. Flujo de datos ..... 7

5. Configuración ..... 8

6. Cómo extender ..... 9

    6.1 Agregar un medio de transporte (ej. patineta eléctrica) ..... 9

    6.2 Agregar paradas o líneas ..... 9

    6.3 Agregar un administrador ..... 9

    6.4 Usar datos oficiales ..... 9

7. Pruebas e integración continua ..... 10

8. Depuración ..... 11

## Información del documento

| Campo | Valor |
|---|---|
| Proyecto | RutaSmart — Asistente inteligente para selección de alternativas de transporte |
| Documento | RS-MT-001 · Manual Técnico |
| Estándar de referencia | IEEE 1016 / IEEE 26514 |
| Versión | 1.0 |
| Autora | Sarah Isabella Peña Cortes (sipenac@academia.usbbog.edu.co) |
| Asignatura | Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles |
| Docente | Ing. Jairo Armando Salcedo Aranda |
| Institución | Universidad de San Buenaventura, Bogotá |
| Fecha | Noviembre de 2026 |

## 1. Propósito

Este manual sirve a quien deba **entender, mantener o ampliar** el código de RutaSmart: estructura, clases principales, flujo de datos, configuración, pruebas, integración continua y cómo extender la app.

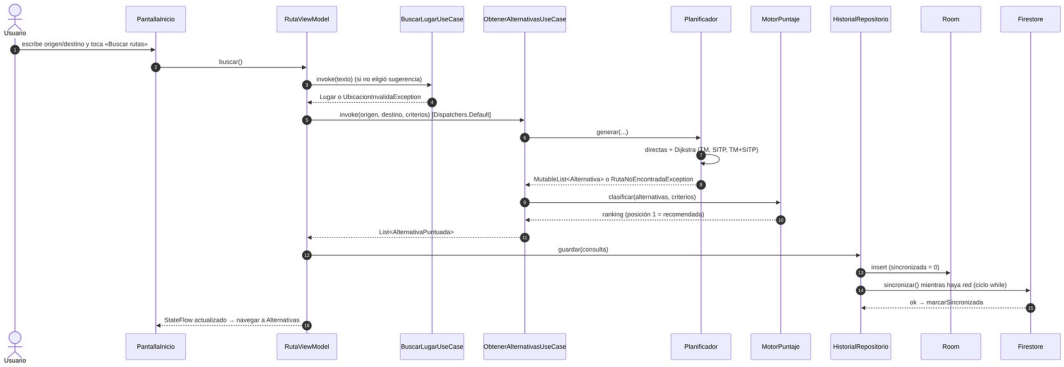
## 2. Estructura del proyecto

```
RutaSmart/
├── app/
│   ├── build.gradle.kts          dependencias, KSP (Room), BuildConfig.GEMINI_API_KEY
│   ├── google-services.json      (lo agrega quien configure Firebase)
│   └── src/
│       ├── main/java/com/sarahpena/rutasmart/
│       │   ├── RutaSmartApp.kt     Application + Contenedor (inyección manual)
│       │   ├── MainActivity.kt
│       │   ├── domain/  model · motor · casosdeuso · ia · admin   (Kotlin puro)
│       │   ├── data/    local (Room) · remoto (Firebase, Gemini) · repositorio · red
│       │   └── ui/      ViewModels · navegacion · pantallas · tema
│       └── test/java/com/sarahpena/rutasmart/   80 pruebas JUnit
├── firebase/firestore.rules
├── azure-pipelines.yml
└── Documentacion/
```

## 3. Componentes principales

| Clase | Responsabilidad |
|---|---|
| Contenedor | Crea una sola vez la base de datos, los repositorios, los casos de uso y la red. |
| SesionViewModel | Login, registro, recuperación, invitado y cierre de sesión; descarga datos de la nube al entrar. |
| RutaViewModel | Estado de la búsqueda (RutaUi), criterios (StateFlow desde Room), historial, buscar(), repetir(), eliminar(). |
| AsistenteViewModel | Mensajes del chat, validación de entrada y llamada al AsistenteRepositorio. |
| BuscarLugarUseCase | Sugerencias y resolución de texto → Lugar (sin tildes); lanza UbicacionInvalidaException. |
| ObtenerAlternativasUseCase | Valida nulos, llama al Planificador y al MotorPuntaje, registra en la Bitácora. |
| Planificador | Alternativas directas + Dijkstra sobre RedTransporte para TM, SITP y TM+SITP. |
| MotorPuntaje | Rangos, puntaje 0–100, ranking y selección con acumulador. |
| HistorialRepositorio | Room primero; sincronización con Firestore (ciclo while); descarga al iniciar sesión. |
| AsistenteRepositorio | Respuesta local + Gemini, límite por minuto y traducción de errores. |
| FirebaseRemoto | Auth y Firestore; espera tareas con Tasks.await en Dispatchers.IO; traduce errores. |
| GeminiClient | POST HTTPS a generateContent con la clave en el encabezado x-goog-api-key. |
| AdminViewModel | Panel de administración: carga el resumen, suspende/reactiva líneas y publica o retira avisos. |
| ServicioRepositorio | Lee y publica config/servicio; aplica las líneas suspendidas al ObtenerAlternativasUseCase; calcula el resumen de uso. |
| AdminValidador / Estadisticas | Reglas del aviso y del estado (mismas que firestore.rules) y cálculo del resumen con HashMap de frecuencias. |

## 4. Flujo de datos




*Secuencia de una búsqueda*

**Concurrencia:**

- **Dispatchers.Default:** cálculo de rutas (CPU).
- **Dispatchers.IO:** red y Firebase.
- **Room:** gestiona sus propios hilos en las funciones suspend.

El estado se expone con StateFlow y Compose lo observa con collectAsState().

## 5. Configuración

| Elemento | Dónde | Notas |
|---|---|---|
| Clave de Gemini | local.properties → GEMINI_API_KEY | Gradle la pasa a BuildConfig; no se sube a Git. |
| Firebase | app/google-services.json | El plugin se aplica solo si el archivo existe. |
| Modelo de IA | domain/ia/AiCore.kt → AiConfig.MODEL | gemini-3.5-flash-lite. |
| Tarifas | domain/model/ MedioDeTransporte.kt → Tarifas | Valores de referencia en COP. |
| Límites del motor | domain/motor/Planificador.kt (companion) | Distancias máximas de caminata, bicicleta y transbordo. |
| Red de transporte | data/red/RedBogota.kt | Paradas, líneas y lugares. |
| Administradores | Administradores.CORREOS (admin@rutasmart.com) o Firestore → admins/{uid} | El correo también está en esAdmin() de firestore.rules; admins solo se crea desde la consola. |

# 6. Cómo extender

## 6.1 Agregar un medio de transporte (ej. patineta eléctrica)

1. Agregar el valor a TipoMedio.
2. Crear una clase que implemente MedioDeTransporte.
3. Agregarla a Medios.todos() y a Medios.de().
4. Si es un medio directo, agregarla en Planificador.generar(); si es público, usarla en nuevas Linea.
5. Asignarle un color en colorDe() (Tema.kt).

## 6.2 Agregar paradas o líneas

En RedBogota:

1. Agregar la Parada al arreglo PARADAS.
2. Crear la Linea y registrarla con red.agregarLinea(linea, listaDeIds). La función valida que las paradas existan.

## 6.3 Agregar un administrador

- **Por correo:** agregar el correo a Administradores.CORREOS (domain/admin/Administracion.kt) y a la función esAdmin() de firestore.rules.
- **Sin recompilar:** en Firebase Console → Firestore → colección admins → documento con ID = uid del usuario y campo rol: "admin".

## 6.4 Usar datos oficiales

Implementar un lector GTFS (stops.txt, routes.txt, stop_times.txt) que construya un RedTransporte, y pasarlo a ObtenerAlternativasUseCase. El motor no cambia.

## 7. Pruebas e integración continua

- **Pruebas locales:** ./gradlew testDebugUnitTest ejecuta las 80 pruebas.
- **Azure Pipelines** (azure-pipelines.yml), en cada cambio a main/develop y en cada Pull Request:
    1. Lint (lintDebug).
    2. Pruebas unitarias.
    3. Generación del APK de depuración, publicado como artefacto.

## 8. Depuración

- **Logcat:** filtrar por RutaSmart/ (solo en compilaciones de depuración). Etiquetas: Rutas, Sync, Prefs, IA, Sesion, Servicio.
- **Puntos de interrupción sugeridos:** Planificador.generar, MotorPuntaje.clasificar, HistorialRepositorio.sincronizar, AsistenteRepositorio.responder.
- **App Inspection → Database Inspector:** ver en vivo las tablas consultas y preferencias.